

Technical Challenges & Solutions Document

Candidate: Sudhir Singh | **Role:** Full Stack Engineer (Internship cum Full Time Employment)

Company: OctaByte AI Pvt Ltd (8byte) | **Case Study:** Dynamic Portfolio Dashboard

Executive Summary

During the development of the Dynamic Portfolio Dashboard, three key technical challenges were encountered when integrating live financial data from unofficial sources (Yahoo Finance and Google Finance), handling public API rate limits, and ensuring real-time UI responsiveness.

This document outlines the top 3 genuine technical challenges faced during the assignment and the engineering solutions implemented.

1. Handling Unofficial APIs, CORS, and Rate Limits (Yahoo & Google Finance)

Problem Description:

Neither Yahoo Finance nor Google Finance provides a free, official public REST API for client applications. Direct browser calls trigger CORS blocks and public IP rate limiting (HTTP 429) when polling 29 stock quotes every 15 seconds.

Applied Solution:

- **Server API Proxy:** Built Node.js Next.js Server API Routes (/api/portfolio and /api/quote) that proxy external requests and bypass browser CORS.
- **In-Memory TTL Caching:** Implemented a server-side 15-second cache for Yahoo CMP quotes and a 60-second cache for Google Finance fundamentals, reducing external HTTP requests by over 95%.
- **1.5s Timeout Safeguard:** Added an AbortController timeout (1500ms) to external fetches. If public financial servers slow down or time out, the backend instantly responds with fallback simulation data in <20ms, preventing UI freezing.

2. Data Accuracy & Anomalous Price Sanity Validation

Problem Description:

Public scraping endpoints occasionally return anomalous values (such as total market cap or daily trading volume instead of unit CMP for certain BSE scrip codes like 541557.BO), causing sudden multi-billion rupee spikes that distort portfolio valuation and charts.

Applied Solution:

- **Price Sanity Validation:** Implemented a server-side price validation filter in src/lib/finance/yahoo.ts checking raw market prices against baseline historical range (0.1x to 4.0x of buy price).
- **Graceful Baseline Fallback:** Any quote failing sanity checks is automatically rejected and replaced with valid price simulation, ensuring 100% data integrity across all charts and portfolio metrics.

3. Real-Time 15-Second Refresh Without UI Flickering or Input Lag

Problem Description:

Auto-refreshing 29 stock holdings every 15 seconds while users are actively typing in the search bar, filtering sectors, or sorting table columns can cause state flickering, re-render lag, and memory leaks.

Applied Solution:

- State Synchronization & Memoization: Utilized `@tanstack/react-table` paired with React `useMemo` hooks for column definitions and sector calculations to isolate component re-renders.
- Lifecycle Cleanup: Scoped `setInterval` timers inside `useEffect` with proper cleanup returns to prevent memory leaks.
- Non-Intrusive Visual Cues: Applied smooth CSS keyframe pulse animations (flash-up / flash-down) to highlight updated price rows without disrupting user interaction.